

Développement d'une e-boutique en PHP (architecture MVC)

Contexte

Vous êtes chargé de développer une application web de type e-boutique permettant à des utilisateurs de consulter des produits, de les ajouter à un panier et de gérer leur compte.

Cette application devra également permettre à des utilisateurs ayant un rôle spécifique (vendeur) de proposer leurs propres produits à la vente via une interface dédiée.

Le projet devra être réalisé en PHP, en respectant une architecture MVC (Modèle / Vue / Contrôleur), similaire à celle étudiée en cours.

L'objectif est de mettre en œuvre vos compétences en développement web, en structuration de projet et en gestion des données.

Objectifs du projet

Votre application devra permettre :

- d'afficher une liste de produits
- de gérer deux types d'utilisateurs : client et vendeur
- de permettre à un vendeur d'ajouter des produits
- de permettre à un client d'ajouter des produits à un panier
- de calculer le total du panier
- d'afficher les informations du profil utilisateur

L'interface graphique devra rester simple et fonctionnelle. L'accent est mis sur la logique et l'organisation du code.

Contraintes techniques

Le projet devra respecter les contraintes suivantes :

- Utilisation de PHP avec PDO pour la base de données
- Architecture MVC obligatoire
- Séparation claire entre :

- modèles (accès aux données)
- contrôleurs (logique de traitement)
- vues (affichage)
- Utilisation de sessions pour la gestion des utilisateurs connectés
- Sécurisation minimale des données (requêtes préparées, mot de passe hashé)

Le front-end doit rester minimal (HTML + CSS simple). Aucun framework JavaScript n'est requis.

Fonctionnalités attendues

1. Catalogue de produits

Une page principale devra afficher une liste de produits.

- Vous devez créer au minimum **10 produits**
- Ces produits doivent être répartis dans **5 catégories différentes**, par exemple :
 - T-shirt
 - Casque
 - DVD
 - Livre
 - Aspirateur

Chaque produit devra contenir :

- un nom
- une description
- un prix
- une catégorie
- une image

Les produits doivent être stockés en base de données.

2. Gestion des utilisateurs

L'application doit gérer deux types de comptes :

- **Client**
- **Vendeur**

Chaque utilisateur doit pouvoir :

- créer un compte
- se connecter
- accéder à une page de profil

La page de profil devra afficher au minimum :

- le nom d'utilisateur
- l'email
- le rôle (client ou vendeur)

3. Gestion des rôles (client / vendeur)

Le système doit différencier les droits selon le rôle :

- Un **vendeur** peut accéder à une page lui permettant d'ajouter des produits
- Un **client** ne doit pas pouvoir accéder à cette page

Vous devrez donc mettre en place une restriction d'accès basée sur le rôle de l'utilisateur.

4. Ajout de produits (espace vendeur)

Un vendeur doit pouvoir accéder à une page contenant un formulaire permettant :

- d'ajouter un produit
- de renseigner :
 - nom
 - description
 - prix

- catégorie
- image

L'image devra être envoyée depuis l'ordinateur de l'utilisateur et stockée sur le serveur.

Des vérifications doivent être mises en place :

- type de fichier (image uniquement)
- taille maximale raisonnable

5. Panier

L'utilisateur doit pouvoir ajouter des produits à un panier.

Le panier devra :

- afficher la liste des produits sélectionnés
- afficher le prix de chaque produit
- afficher un **total général**

Le panier peut être stocké en session pour simplifier la gestion.

6. Calcul du total

Dans la page panier, vous devez afficher :

- le total de tous les produits ajoutés

Le calcul doit être dynamique en fonction du contenu du panier.

Organisation attendue (MVC)

Votre projet devra être organisé de la manière suivante :

- **Models** : fonctions liées à la base de données (produits, utilisateurs, panier)
- **Controllers** : gestion des actions (connexion, ajout produit, panier, etc.)
- **Views** : affichage HTML

Aucune requête SQL ne doit être écrite dans les vues.

Aucun traitement de formulaire ne doit être effectué directement dans les vues.

Éléments à prévoir en base de données

Vous devrez concevoir une base contenant au minimum :

- une table users
- une table products
- une table categories
- éventuellement une table pour le panier si vous ne passez pas par la session

Conseils méthodologiques

Avant de commencer à coder, il est fortement recommandé de réfléchir à l'organisation du projet.

Posez-vous les questions suivantes :

- Quelles sont les pages nécessaires ?
- Quelles actions doivent être gérées par le contrôleur ?
- Quelles données doivent être stockées en base ?
- Quels éléments doivent être affichés dans les vues ?

Pour la gestion des rôles, vous pouvez vous appuyer sur la session utilisateur.

Par exemple, vous pouvez stocker le rôle lors de la connexion et vérifier ce rôle avant d'afficher certaines pages.

Pour cela, vous pouvez rechercher dans la documentation PHP :

- l'utilisation de `$_SESSION`
- les conditions en PHP (if, isset)
- la gestion des accès conditionnels

Pour l'upload d'image, vous pouvez vous référer à :

- \$_FILES
 - move_uploaded_file()
 - la vérification des extensions avec pathinfo()
-

Attendus

À la fin du projet, votre application devra :

- afficher des produits organisés par catégorie
 - permettre la connexion et l'inscription
 - différencier les rôles utilisateur
 - permettre à un vendeur d'ajouter des produits
 - permettre à un client d'ajouter des produits au panier
 - afficher correctement le total du panier
 - respecter une architecture MVC propre et cohérente
-

Critères d'évaluation

Vous serez évalué sur :

- la structure MVC du projet
- la qualité du code (lisibilité, organisation)
- le respect des fonctionnalités demandées
- la gestion des données (PDO, requêtes préparées)
- la gestion des rôles
- le bon fonctionnement global de l'application